

Fundamentos de Inteligencia Artificial

- Agente Connect-4: Flat Monte Carlo -

1. Idea principal y diferencia respecto al grupo

El agente implementado es **Flat Monte Carlo (FMC)**: dado el estado actual del tablero, estima el valor de cada columna legal ejecutando N simulaciones aleatorias (*rollouts*) desde esa jugada hasta el final de la partida. Elige la columna con mayor tasa de victoria estimada.

Lo que lo diferencia de los otros agentes del grupo:

- Es completamente **online**: no tiene fase de entrenamiento ni memoria entre partidas. Toda la inteligencia emerge en tiempo de decisión.
- Su único parámetro es N (rollouts por acción), lo que lo hace totalmente transparente y permite analizar su comportamiento como función de un solo tornillo numérico.
- Escala naturalmente con el tiempo disponible: más budget = mayor N = mejor calidad, sin necesidad de reentrenar ni guardar nada.

Código final (branch Política-Juan):

<https://github.com/10santiago12/Connect-4-IA/blob/Política-Juan>

2. Versiones evaluadas

Versión	N	Descripción
V1	100	Rápida — decisión en ~ 300 ms
V2	200	Versión final entregada — decisión en ~ 680 ms, mejor calidad vs oponentes fuertes

3. Análisis del agente

El análisis completo está en `entrega.ipynb`. A continuación se presentan las conclusiones de los cuatro experimentos ejecutados.

3a. Configuración del agente: efecto de N

Exp 1 — Win rate vs jugador aleatorio ($N \in \{10, 25, 50, 100, 200\}$, 20 partidas por N): FlatMC alcanza ~ 100 % de win rate contra el aleatorio **desde** $N = 10$, tanto como rojo como amarillo. V1 y V2 son indistinguibles contra este oponente. Conclusión: el agente supera ampliamente el prerequisite de la rúbrica (>50 %) incluso con recursos mínimos.

Exp 2 — Trade-off tiempo / calidad ($N \in \{10, \dots, 1000\}$): El tiempo de decisión crece linealmente con N ($R^2 \approx 1$). V2 (680 ms/decisión) permite más de 85 movimientos dentro del budget de 1 minuto, sin restricción práctica. Solo a partir de $N \approx 500$ el budget empieza a ser un limitante real.

Cuello de botella: FlatMC distribuye N rollouts *uniformemente* entre todas las columnas legales, incluyendo jugadas claramente malas. Con 5–7 columnas en mid-game, la mayoría del presupuesto se desperdicia.

3b. Oponente: el agente contra sí mismo

Exp 3 — Auto-desempeño, heatmap N_a vs N_b (10 partidas por par, ambos colores): Mayor N domina consistentemente. V2 ($N = 200$) gana **100 %** contra $N = 10$ y $N = 50$, y **60 %** contra V1 ($N = 100$). V1 solo gana el 30 % contra V2. En términos de win rate promedio vs todos los oponentes: V2 $\approx$ **77 %** frente al **62 %** de V1 — 15 puntos de diferencia — justificando empíricamente la elección de $N = 200$ como versión final.

Exp 4 — Ventaja de color (diagonal del heatmap + datos del Exp 1): Ambos colores dominan al aleatorio de manera idéntica desde $N = 25$. Cuando ambos agentes tienen el mismo N , los resultados son inconsistentes por muestra pequeña (10 partidas). A $N = 200$ simétrico el resultado es 50/50, confirmando que el desempeño del agente depende de N y no del color asignado.

4. Propuesta de mejora

Problema: los rollouts uniformes entre columnas son ineficientes — el presupuesto N se gasta en jugadas malas igual que en jugadas buenas.

Mejora propuesta: MCTS con UCB1. En lugar de distribuir N rollouts equitativamente, UCB1 guía la exploración hacia las columnas más prometedoras mediante:

$$\text{UCB1}(s, a) = \hat{q}(s, a) + C \cdot \sqrt{\frac{\ln N_{\text{padre}}}{N_a}}$$

El segundo término es un bonus de exploración que disminuye a medida que una acción se visita más. Esto concentra el mismo budget N donde realmente importa.

Por qué resuelve el cuello de botella: Con el mismo $N = 200$, MCTS-UCB1 debería ganarle a FlatMC($N = 200$) en auto-desempeño y alcanzar calidad equivalente a FlatMC($N = 500$) en menos tiempo — predicción verificable ejecutando los mismos experimentos con el nuevo agente.

Mejora secundaria de bajo costo: hacer que dentro del rollout, si un jugador puede ganar en un movimiento lo haga. Rollouts más realistas = estimaciones más precisas, sin aumentar N .